Code Walkthrough (for a teammate joining after the build)

Read in this order: `protocol.py`, `receiver.py`, `sender.py`, `emulator.py`, `cli.py`, then the rest. Line counts are approximate. Run `pytest -q` first to see everything pass.

mpft/protocol.py (about 95 lines): wire format

Frame = 20-byte header (`MP`, version, type, session, chunk index, payload length, CRC-32) plus payload. Key functions: - `pack_header` / `unpack_header`: build and validate a header; raise `ProtocolError` on bad magic, version, type, CRC or oversize length. - `send_msg`, `recv_exact`, `recv_msg`: send one frame; read exactly n bytes from a stream socket (TCP can split data); read one whole frame. - `build_manifest`: one streaming pass over the file producing per-chunk SHA-256 list and whole-file SHA-256. - `chunk_len`, `parse_addr`, `tune` (sets TCP_NODELAY).

Questions: Why does `recv_exact` loop? What does the CRC cover? Why cap `payload_len` at 64 MiB?

mpft/receiver.py (about 160 lines): the verifying end

- `Session`: one transfer. `__init__` opens/creates `name.part`, sizes it, and `_scan` re-hashes existing chunks (resume). `finalize` verifies the whole file and renames. `snapshot` feeds the status file.
- `ReceiverHandle`: `_accept_loop` (one thread per connection), `_serve` (HELLO, then loop over MANIFEST/CHUNK/DONE), `_open_session`, `_store` (validate length + SHA-256, `pwrite`, ACK/NACK, duplicates), `stop`.
- `start_receiver(listen, out_dir)` returns the handle (`.port`, `.stop()`).

Questions: Why is `pwrite` used instead of `seek+write`? What happens on a duplicate chunk? When is the `.part` file renamed?

mpft/sender.py (about 215 lines): scheduler and recovery

- `Sender.connect` (4 tries with backoff, sends HELLO), `ensure_manifest` (first path sends MANIFEST, others wait; fills the queue from MANIFEST_ACK).
- `worker`: per-path loop: connect, pull from queue, `send_chunk`, handle failures with backoff.
- `send_chunk`: `pread` chunk, optional bit flip (`corrupt_rate`), send, wait for ACK/NACK, record RTT.
- `on_ack`, `fail_path`, `requeue`: bookkeeping, recovery time, retry limit and backoff via `threading.Timer`.
- `finish`: DONE handshake. `snapshot`: status data. `run`: start threads, wait, finish, build result.
- `run_transfer(...)`: public entry; returns the metrics dict.

Questions: How does a faster path end up carrying more bytes? Why do retries use priority 0? What happens when a socket times out mid-chunk (why close it)?

mpft/metrics.py (about 15 lines)

`build_result` turns sender counters into `total_time_s`, `goodput_mbit`, `bytes_per_path`, RTT mean and jitter (stdev), `recovery_time_s`, `success`. Questions: How is goodput computed after a resume? What is jitter here?

mpft/status.py (about 50 lines)

`StatusWriter` thread writes `status/<role>-<session>.json` every second (temp file + `os.replace`), derives per-path Mbit/s from byte deltas. Questions: Why an atomic replace? Where does throughput come from?

mpft/emulator.py (about 130 lines): fake network paths

`PathProxy` listens on a port and relays to the receiver. `_reader` stamps blocks with a due time (delay, jitter) and may flip a bit; `_writer` waits until due, sends, then sleeps to enforce `rate_mbit`. `kill` closes the listener and every socket (the `drop_after_s` timer starts at the first connection). `start_emulator(config)` / `run_emulator(path)`. Questions: Why separate reader and writer threads? How is reordering avoided? Why does `drop_after_s` leave the port refusing connections?

mpft/cli.py (about 80 lines)

`main` builds argparse subcommands `send`, `receive`, `emulate`, `dashboard`; `cmd_send` expands `--paths N` into N targets and can append a CSV row; `--corrupt-rate` is hidden. Questions: What does `--to` with one address and `--paths 4` do? How would you add a flag?

web/ (dashboard)

`app.py`: `/api/status` reads status files, marks stale. `static/app.js` polls and renders with `textContent` (no HTML injection). `style.css`: the plain theme. Questions: Why mark files stale? Why `textContent` instead of `innerHTML`?

experiments/

`run_experiments.py`: runs receiver and emulator in-process, one function per experiment, writes `results/*.csv` (resume uses a subprocess that is killed with SIGKILL). `plot_results.py`: matplotlib PNGs. Questions: How is the resume experiment interrupted? Why one run per setting is weak evidence?

tests/

`test_protocol.py` (framing), `test_transfer.py` (20 MB loopback, NACK, duplicates), `test_failure.py` (killed path, silent path, corruption, retry limit, resume), `test_emulator.py`, `test_dashboard.py`.

`conftest.py` has the `receiver` fixture and a per-test status directory. Questions: How does the `FlakyProxy` kill a path deterministically? Why do tests use random but seeded data?